



Cardiac Companion 4.0 · Android · Google Play only

Work order for Claude Code

From the version 3.2 web app to an Android app distributed only through Google Play – phased, testable, and safe for the people who will rely on it.

Repository: aminmohammed2k7-igtm/cardiac-companion · 3 October 2026

The short version

Wrap the existing app in Capacitor 8 (Android only). Keep every rule and safety limit. Replace browser features with native ones: reminders that fire when the app is closed, durable storage, Android file sharing. Remove web-only parts. Keep the camera pulse out of the public build. Prepare everything Google Play needs.

Ten phases (0–9), one at a time. Each ends with tests on Amin's phone before the next one starts.

Part A · For Amin

Laptop setup, how to run each phase, the decisions only you can make.

Part B · For Claude Code

Mission, safety charter, architecture, phases 0–9, device tests, definition of done.

Appendices

CLAUDE.md to create, prompts to paste, calendar test vectors, your Google Play checklist.

Contents

Part A – For Amin: read this first	3
A1. Set up your laptop	3
A2. Where to run Claude Code	3
A3. How to run each phase	3
A4. Decisions only you can make	4
A5. Jobs that stay with you	4
Part B – The order (addressed to Claude Code)	5
B1. Mission	5
B2. What exists today	5
B3. Safety charter – never break these	5
B4. Architecture decisions (already made)	5
B5. How to work	6
B6. The phases	6
Phase 0 – Baseline and audit	6
Phase 1 – A safety net for the medical logic	7
Phase 2 – Android shell with Capacitor 8	7
Phase 3 – Durable storage	8
Phase 4 – Reminders that work when the app is closed	9
Phase 5 – Files, sharing and calls	10
Phase 6 – Camera pulse: off in the public build	10
Phase 7 – Android polish and accessibility	11
Phase 8 – Release build and Google Play preparation	11
Phase 9 – Retire the website and finish the documentation	12
B7. Device test matrix	13
B8. Definition of done for 4.0	13
Appendix A – CLAUDE.md (create exactly this file)	14
Appendix B – Prompts to paste	15
Appendix C – Test vectors	16
Appendix D – Amin's Google Play checklist	17

Part A — For Amin: read this first

A1. Set up your laptop

- Git, and Node.js 22 (LTS).
- Android Studio (current stable) with the Android SDK for API 36. Let Gradle use Android Studio's bundled JDK 21.
- Claude Code, installed by following the official guide at code.claude.com/docs.
- Your Samsung M14 with Developer options and USB debugging switched on, plus a data cable. Your laptop's 16 GB of RAM is enough. Test on the real phone first; use an Android 16 emulator only for the checks the order names.

A2. Where to run Claude Code

Run Claude Code on your laptop, inside the repository folder, for this whole job — Android builds, signing and phone tests all happen there. Claude Code on the web is fine for Phases 0 and 1, which are only JavaScript and tests.

A3. How to run each phase

1. Copy this PDF into the repository as `docs/cClaude-code-order.pdf` and your Complete App Guide as `docs/app-guide.pdf`. Commit both.
2. Open a terminal in the repository folder and start Claude Code.
3. Paste Prompt 0 from Appendix B. Claude creates `CLAUDE.md` — the permanent rules in Appendix A — and plans Phase 0.
4. Use plan mode for every phase: press Shift+Tab until the status bar shows plan mode, or start your message with `/plan`. Read the plan, then approve it or correct it.
5. When Claude finishes a phase, it stops and gives you step-by-step tests for your phone. Do them. Merge only when they pass, then paste the next phase's prompt.

Claude Code reads PDFs in page ranges, at most 20 pages at a time. Page 2 of this document is the contents page, and every prompt tells Claude to start there.

A4. Decisions only you can make

Decision	Recommended default	Why it matters
Application ID (package name)	A reverse domain you or your partner control, e.g. <code>org.partnername.cardiaccompanion</code> . Letters, digits and dots only.	It can never change after the first upload to Google Play.
Who publishes on Google Play	A partner organization's developer account	Google requires an organization account, with a D-U-N-S number, for health apps such as medical apps.
Camera pulse in the public app	Off	Not yet validated; the camera is a health-sensitive permission; avoids medical-device questions.
Android's automatic cloud backup	Off – people use the app's own backup file	Keeps “nothing leaves the phone” literally true.
Oldest Android version	Android 7.0 (API 24), Capacitor 8's default	Many budget phones in Ethiopia run older Android versions.
Internet permission	Removed, if the app works without it	A privacy promise anyone can verify.
Touch-target size	48 px for every touch target (some are 44 px today)	Android accessibility guidance; Play's pre-launch checks flag smaller targets.
Licence	Apache-2.0 or MIT	Needed before the code is shown publicly.

A5. Jobs that stay with you

Never hand these to Claude:

- Creating and storing the upload signing key and its passwords (keep two offline copies).
- Play Console: the account, identity checks, payments and every declaration.
- Agreeing health wording with clinicians.
- Any agreement with a partner organization.

Part B – The order (addressed to Claude Code)

Claude, this part is your brief. Follow it phase by phase. If anything conflicts, the safety charter in B3 wins. If something is unclear, stop and ask Amin.

B1. Mission

Turn Cardiac Companion v3.2 – an offline, single-file web app for heart patients in Ethiopia – into Cardiac Companion 4.0: an Android app built with Capacitor 8 and distributed only through Google Play. Keep every existing behaviour, rule and safety limit unless this order says otherwise. Replace browser-only features with native Android ones. Remove web-only features. Prepare, but never submit, everything Google Play needs.

B2. What exists today

- Repository `aminmohammed2k7-igtm/cardiac-companion`; web deployment at `cardiac-companion.vercel.app`.
- `index.html` (6,024 lines: layout, stylesheet, every screen, three languages, all logic), `ppg-engine.js` (1,601 lines, camera pulse engine), `sw.js` (72 lines), `pulse-lab.html` (developer tool), `CHANGES.md`, and 90 Node tests in `tests/` (14 look, 49 engine, 27 camera mock).
- Unmerged branch `claude/wizardly-bardeen-oxape2`: restores the camera pulse button and panel that were missing from the page.
- `docs/app-guide.pdf`: the complete guide to v3.2, with a file-by-file line index. Read it before Phase 0.

B3. Safety charter – never break these

1. Never compute, suggest, round or adjust any dose. Show only what the user entered from their doctor.
2. No interpretation beyond what v3.2 does: INR against the doctor's range, the fixed warning-sign texts, and the documented weight, symptom and mood rules.
3. Health data never leaves the phone, except files the user explicitly chooses to share. No servers, accounts, analytics, crash reporting, ads, remote configuration, remote fonts or remote code. No new network calls of any kind.
4. Health wording is frozen in all three languages. Don't add, delete or reword it without Amin's explicit approval; put every proposed change in `docs/NEEDS-CLINICAL-REVIEW.md`.
5. Never lose user data. Any storage change needs a tested, lossless migration and keeps the previous copy until Amin removes it.
6. Dates use the phone's local calendar day, never UTC.
7. The camera pulse is off in release builds, and the release has no CAMERA permission.
8. If you suspect a bug in medical logic, don't fix it silently: write it in `docs/android/FOUND-BUGS.md` and ask Amin.
9. Every phase ends with all tests green.

B4. Architecture decisions (already made)

- Capacitor 8 – the latest stable 8.x when you start; check npm – with the Android platform only. No iOS project.
- Web code moves into `www/` (Capacitor's `webDir`) and stays plain HTML, CSS and JavaScript: no framework, no bundler.
- Medical logic is extracted into plain JavaScript modules that run in both the `WebView` and `Node`, following the dual pattern `ppg-engine.js` already uses, so it can be tested.
- Every browser-only feature goes through a small platform layer in `www/js/platform/`: storage, reminders, files and sharing, clipboard, device info, app lifecycle. UI code calls the platform layer, never a Capacitor

plugin directly. A browser fallback is allowed for development and tests only; it is never shipped as a public website.

- Official @capacitor/* plugins first. Any community plugin needs Amin's OK: name, purpose, licence, last release date and weekly downloads.
- Fonts (Onest and Noto Sans Ethiopic) are bundled inside the app with their OFL licence files.

B5. How to work

- One phase per branch (phase-N-short-name). Plan first; wait for Amin's approval.
- Small commits. Run `npm test` before every commit. Never skip or weaken a test to make it pass.
- Ask before: adding any dependency, changing the application ID, changing the storage format, removing a feature, changing a design rule, or touching signing.
- Never read, print or commit keystores, passwords or `keystore.properties`.
- Amin must be able to explain every decision himself, so explain your reasoning in plain English and comment the why, not just the what.
- End every phase with the report that phase asks for.

B6. The phases

Each phase lists its goal, its tasks, what “done” means, and what to report. Do them in order.

Phase 0

Baseline and audit

Goal. Know exactly what exists before changing anything. No behaviour changes in this phase.

Tasks

1. Review branch `claude/wizardly-bardeen-oxape2` and summarise its diff for Amin. If he approves, merge it into main and tag the result `web-v3.2.1`.
2. Run all three test files and record the counts (expected $14 + 49 + 27 = 90$).
3. Create `docs/android/API-MAP.md`: every browser-only API the app uses, with line numbers, its planned Android replacement and the phase that replaces it. Check at least: `localStorage`; the Notification API and service-worker notifications; service worker registration; the install prompt and “Add to home screen”; `navigator.storage.persist`; `navigator.share` and `canShare` with files; `navigator.clipboard`; download links and Blob URLs; file inputs; the screen wake lock; `getUserMedia` with torch and exposure constraints; cross-tab storage events; `visibilitychange`; `matchMedia` (dark mode, reduced motion); the Intl Islamic (Umm al-Qura) calendar; Google Fonts; tel: links; the runtime web-app manifest.
4. Create `docs/android/BASELINE.md`: version, test counts, file sizes, and every screen and panel (from the guide) as a regression checklist.

Done when

No app code has changed except the approved merge, and both documents exist.

Stop and report: the diff summary, the highlights of the API map, and any risk you see.

Phase 1**A safety net for the medical logic**

Goal. Pin today's behaviour with tests before anything moves.

Tasks

1. Extract pure functions (no DOM, no storage) from `index.html` into modules in `js/core/` next to it (they move into `www/` with everything else in Phase 2): calendar (Ethiopian date, Ethiopian clock, local date keys), seasons (Orthodox Easter, Abiy Tsom, Ramadan), doses (daily plans, freezing, adherence), supply, warfarin (tablet maths, INR status, time in range, dose history), injection, and health rules (weight gain, PHQ-2, severe symptoms). Move code verbatim; the app calls the modules; behaviour stays identical.
2. Write tests in `tests/core/` in the existing style (plain Node, exit code 1 on failure). Cover every rule in the guide's "Features and logic" section, including: Pagume 6 in Ethiopian leap years; New Year on 12 September before a Gregorian leap year; the 23:59 → 00:00 rollover; days the app wasn't opened; every-other-day counted from the start date; weekday schedules; "when needed"; a medicine added or removed mid-period; a warfarin day of 0 versus blank; the quarter-tablet rule; supply warnings firing once; Rosendaal time in range with a gap over 120 days and with less than 7 days of line; penicillin completion with the 3-day grace; the weight rule against each of the previous 1–3 days; the PHQ-2 threshold of 3; and the reminder windows (first at most 3 hours late, second at most 4 hours late).
3. Use the test vectors in Appendix C.
4. Add `tests/i18n.test.js`: every key exists in `en`, `am` and `or`, with no empty strings.
5. Create `package.json` (private, no dependencies) so that `npm test` runs every test file and fails if any fails.

Done when

All old and new tests pass, and a manual check in a desktop browser shows no visible change.

Stop and report: the new test count, and anything surprising (suspected bugs go into `FOUND-BUGS.md` – don't fix them).

Phase 2**Android shell with Capacitor 8**

Goal. The existing app runs as an installed Android app, fully offline, with the web-only parts removed.

Tasks

1. Install Capacitor 8 (@capacitor/core, @capacitor/cli, @capacitor/android; latest 8.x). Move the app into `www/`. Move `pulse-lab.html` to `tools/` – it is a developer tool and is not shipped. Update test paths.
2. Run `npx cap init` with app name "Cardiac Companion" and the application ID Amin chose, then `npx cap add android`. Keep Capacitor 8's defaults: `minSdk 24`, `compileSdk` and `targetSdk 36` (Google Play has required API 36 for new apps since 31 August 2026).
3. Bundle the fonts: download Onest and Noto Sans Ethiopic from the official google/fonts repository into `www/fonts/` with their OFL licence files, and replace the Google Fonts links with `@font-face`. Keep the same fallback stack.
4. Remove web-only features from the app: the service worker and its registration, the runtime manifest, the install prompt and "Add to home screen", the persistent-storage request and its Settings line, and cross-tab sync. List every removal in `CHANGES.md`.
5. Edge-to-edge is forced on Android 16 for apps targeting API 36. Use `viewport-fit=cover` and the `--safe-area-inset-*` CSS variables that Capacitor's SystemBars injects, so the header and the bottom tab bar clear the status and navigation bars. Style the bars for light and dark themes.
6. Back button and predictive back: close the top panel → close an open form → return to Today → leave the app. Use @capacitor/app's `backButton` event, and verify on an Android 16 emulator with gesture navigation (apps targeting API 36 no longer get the old back callback).

7. Font size: check how Android's system font size affects the WebView on Amin's Samsung (Settings → Display → Font size). Recommended: fix the WebView text zoom at 100% and, on first launch only, switch on the app's own Large text when the system font scale is 1.15 or more. Explain your choice.
8. Large screens: Android 16 ignores portrait locks on tablets for apps targeting API 36. Check the wide-screen rail layout on a tablet emulator in both orientations.
9. Icons and splash: an adaptive launcher icon from the 512 px maskable icon; a monochrome (white on transparent) heartbeat icon for notifications; a plain #EEF1EE splash with the logo.
10. Privacy in the manifest: `android:allowBackup="false"` plus `dataExtractionRules` that exclude everything from cloud backup and device transfer, unless Amin chose otherwise. Then try removing the INTERNET permission from the release build (`tools:node="remove"`). If everything works, keep it removed; if not, keep it and explain why in `docs/android/PERMISSIONS.md`.
11. Add a GitHub Actions workflow that runs the tests and builds a debug APK on every push, keeping the APK as a downloadable artifact.

Done when

The debug build installs on the Samsung M14 from Android Studio; with airplane mode on from the very first launch, every tab and panel works (apart from features that later phases replace); there are no errors in `chrome://inspect`; and all tests pass.

Stop and report: what to tap to check each screen, the APK size, and the final permission list from the merged manifest.

Phase 3

Durable storage

Goal. The record survives everything except uninstalling, with no size limit and no half-written saves.

Why. `localStorage` inside a `WebView` has a quota of a few MB (years of dose logs for several people can reach it), no atomic writes and no recovery copy.

Tasks

1. Add `www/js/platform/storage.js`. Keep the synchronous read API the app uses today (an in-memory cache loaded once at start-up) and persist asynchronously through `@capacitor/filesystem` in the app's private data folder: one JSON file per person, plus profiles, global and preferences files.
2. Atomic saves: write to `*.tmp`, then rename over the real file, and keep the previous version as `*.bak`. On load, validate with the existing repair-on-load; if the main file is damaged, fall back to the `.bak` copy and tell the user calmly that this happened.
3. Save shortly after each change (a short debounce) and always flush when the app goes to the background.
4. Migration: on the first start of 4.0, if the `WebView`'s `localStorage` holds `cc-*` keys, copy them into files, verify by reading back and comparing, and keep the `localStorage` copy – don't delete it.
5. Restoring a v3.x web backup file must still work.
6. Tests with a fake filesystem in Node: normal save and load, a crash between writing the `.tmp` file and the rename, a damaged main file, a missing file, the migration, and a backup → restore round trip compared deep-equal.

Done when

Swiping the app away right after marking a dose and reopening it loses nothing; backup → restore gives identical data; switching between two people works.

Stop and report: the storage layout, the migration steps and the test results.

Phase 4**Reminders that work when the app is closed**

Goal. Replace the in-page reminder engine, which only worked while the page was open, with native scheduled notifications.

Tasks

1. Before writing code, write docs/android/REMINDERS.md and wait for Amin's OK. It must explain how reminders keep firing even if the app isn't opened for weeks – for example repeating schedules where the content never changes from day to day, rolling one-off schedules where it does, and a gentle “Open the app to keep your reminders running” notice at the end of any rolling window – and how the total stays well under Android's per-app alarm limit (around 500).
2. Use @capacitor/local-notifications and keep v3.2's rules: one notification per person per dose time (“Medicines at 08:00 – 3 to take”); medicine names only if the person ticked “Show medicine names in reminders”; one follow-up an hour later, cancelled as soon as all those doses are marked; warfarin only on days with a dose above 0; and the daily “Coming up” notice after 07:00 for the injection, the INR test and the clinic visit. Never notify about something the user just did.
3. Stable IDs (a hash of person, date, time and kind) so rescheduling is idempotent. Reschedule on: app start, return to the foreground, any change to medicines, warfarin, injection, visits, settings or people, marking or undoing a dose, day rollover, a time or time-zone change, and restore.
4. Doze: use allowWhileIdle. It can fire at most about once every 9 minutes per app, so doses due at the same time must stay in one notification.
5. Exact timing: declare SCHEDULE_EXACT_ALARM only. Do not declare USE_EXACT_ALARM – Google Play restricts it to alarm-clock and calendar apps. On Android 14+ exact alarms are off by default for new installs: check with checkExactNotificationSetting(), explain in plain words why on-time reminders matter, and offer changeExactNotificationSetting(). If the user declines, keep working with inexact timing and say in Settings that reminders may arrive a few minutes late.
6. Notification permission (Android 13+): ask right after the first medicine is added, with a one-line reason – not at first launch. If refused, show a calm banner on Today with a button to the settings.
7. Channels: “Medicine reminders” (high importance) and “Coming up” (default), with lock-screen visibility set to private.
8. Tapping a notification opens Today for the right person.
9. A Settings page, “Make reminders reliable”, with short steps for Samsung and for TECNO, itel and Infinix battery settings, and a button that opens the app's system settings. Don't request the battery-optimisation exemption (REQUEST_IGNORE_BATTERY_OPTIMIZATIONS): Google Play allows it only in narrow cases, and exact alarms already fire in Doze.
10. A “Send a test reminder in 1 minute” button, so the user can lock the phone and check.
11. Keep “Add reminders to my calendar” (.ics) as an optional extra under Reminders.

Done when

On the Samsung M14, a reminder fires with the screen off and the app swiped away; after a reboot; in forced Doze (adb shell dumpsys deviceidle force-idle); with exact alarms denied (late but present); with notifications denied (the banner shows); with two people (names in titles); and the lock screen hides medicine names when that setting is off.

Stop and report: each of those results and how you tested it.

Phase 5 Files, sharing and calls

Goal. Every export and import works through Android's own file and share system.

Tasks

1. Replace download links and `navigator.share` with `@capacitor/filesystem` (write to the cache folder) and `@capacitor/share` (the share sheet) for the backup JSON, the CSV, the visit summary PDF, the medical ID PDF and the calendar file. Keep the file names.
2. Restore: first test a file input inside the WebView on Amin's phone; if it's unreliable, propose a file-picker plugin. Validate the file before replacing anything, ask for confirmation, then reload state through the storage layer.
3. Clipboard through `@capacitor/clipboard` for “Send to family”.
4. Call buttons open the dialer with the number filled in (tel:) and never call directly. No `CALL_PHONE` permission.
5. Delete exported temporary files older than one day at start-up.

Done when

Each export can be sent to Telegram and opened; a v3.2 web backup restores correctly; PDFs show Amharic text correctly.

Stop and report: the test results and any file-picker decision needed.

Phase 6 Camera pulse: off in the public build

Goal. The public app has no camera feature and no `CAMERA` permission; the engine stays available for research.

Tasks

1. Add a build-time feature flag, `cameraPulse`. Release: `false` – no button, no panel, no reachable camera code path. Debug builds may set it to `true`.
2. Make sure the release manifest has no `CAMERA` permission (add it only to the debug manifest if needed), and prove it by listing the release build's permissions.
3. Keep `ppg-engine.js` and its 76 tests (49 + 27) passing, and keep `tools/pulse-lab.html` working in Chrome for Amin's validation study.
4. Only if Amin asks later (Phase 6b): in a debug build, test `getUserMedia` with the torch inside the WebView on the Samsung and on one TECNO, itel or Infinix phone. If the torch can't be controlled, propose a small native frame source (CameraX with the torch on, the average colour of the frame centre with a timestamp for each frame) that feeds the existing `SignalProcessor`. Any camera feature needs a clear explanation screen and the user's consent before the permission request, which Google Play expects for health-related sensitive permissions such as the camera.

Done when

The release build's permission list has no `CAMERA`, and all tests pass.

Stop and report: the release permission list and the test counts.

Phase 7**Android polish and accessibility**

Goal. The app feels native, readable and usable for older and unwell people on budget phones.

Tasks

1. Touch targets: raise every touch target to at least 48 px, then update `tests/look.test.js` – after Amin approves the design-rule change.
2. A TalkBack pass on every tab and panel: roles, labels, focus order and announcements in all three languages, with the page language switching between en, am and om.
3. Dark mode, reduced motion and Large text checked on the phone.
4. Keyboard: the weight, INR and reading fields stay visible above the keyboard, and Enter still saves.
5. A calm error screen if something unexpected breaks: “Something went wrong. Your record is safe.” with a Copy details button. No network.
6. Performance: measure cold start and tab switching on the Samsung M14 and, if possible, on a 2–3 GB RAM TECNO, itel or Infinix phone.

Done when

The checklist in `docs/android/A11Y.md` is complete, with screenshots.

Stop and report: the checklist, the timings and anything that still needs work.

Phase 8**Release build and Google Play preparation**

Goal. A signed release bundle builds on Amin's laptop, and every Play Console answer is drafted for him to review. Claude prepares; Amin submits.

Tasks

1. Versioning: `versionName 4.0.0`, and a documented `versionCode` scheme (for example 40000).
2. Signing: give Amin the exact `keytool` command to create the upload key on his own laptop. Read signing details from a git-ignored `keystore.properties`. Never ask for, print or commit passwords or the keystore. Remind Amin to keep two offline copies of the key.
3. Build the Android App Bundle (`gradlew bundleRelease`). Report: `targetSdk 36`, the final permission list, the bundle size, and whether any plugin ships native libraries – if one does, confirm 16 KB page-size support with Android Studio's APK Analyzer.
4. A demo profile for screenshots: a clearly fictional “Demo Patient”, never real data. Produce phone screenshots in English and Amharic (Today, Medicines, Summary, Medical ID), a 1024 × 500 feature graphic and a 512 px icon.
5. Drafts in `docs/play/` for Amin to review – submit nothing: the store listing in English and Amharic (short description up to 80 characters, full description up to 4,000); the privacy policy in plain English and Amharic; Health apps declaration answers; Data safety answers based on what the code actually does; content rating answers; target audience (adults); app access (no login needed); ads (none).
6. An “About and licences” screen: version, publisher, developer credit, the disclaimer, the privacy policy link, and open-source licences (Capacitor: MIT; fonts: OFL).

Listing rules (from Google Play's health policy)

The full description must include this sentence: “Cardiac Companion is not a medical device and does not diagnose, treat, cure, or prevent any medical condition.” Also tell people to follow their doctor's or clinic's advice.

Health apps declaration: Medical → Medication and treatment management, and Diseases and conditions management. Not a medical device app.

Never use claims the app can't support: no “measures”, “monitors your heart”, “diagnoses”, “AI” or “smart”.

Done when

A signed release bundle builds on Amin's laptop, and all drafts exist.

Stop and report: the bundle details and a short list of what Amin must review before submitting.

Phase 9**Retire the website and finish the documentation**

Goal. The web version stops being a way to use the app, existing web users can move their data, and the repository explains itself.

Tasks

1. Website (Vercel): replace the root with a simple static project page — what the app is, its status, a privacy policy page, contact details, and the Google Play link once the app is live.
2. Keep the old web app reachable at /legacy/ on the same domain for a transition period, with a banner: “This web version is retiring. Tap Back up now, then restore the file in the Android app.” The same domain matters because the old data lives in each browser's storage for that domain. The legacy copy must not register a service worker.
3. Replace /sw.js with a small service worker that deletes its caches and unregisters itself, so phones stop showing the cached old app at the root.
4. Documentation: README.md (what it is, the safety charter, screenshots, how to build and test, licence); a 4.0 entry in CHANGES.md listing everything removed and everything in NEEDS-CLINICAL-REVIEW.md; docs/ARCHITECTURE.md (layers and data flow); and an updated app guide.

Done when

The project page is live, the legacy backup path has been tested in a phone browser, and the documents are updated.

Stop and report: links to the page and the documents, and anything left open.

B7. Device test matrix

Device	Why
Samsung Galaxy M14 (Amin's phone)	Main real device; Samsung's own battery management.
Any TECNO, itel or Infinix phone (borrowed)	The budget brands many target users own; aggressive battery savers.
Android 16 emulator (API 36), gesture navigation	Predictive back and edge-to-edge.
Small-screen emulator (360 × 640)	Layout at the minimum width.
Tablet emulator, both orientations	Android 16's large-screen rules.

Scenarios to run before calling 4.0 done:

- ☐ First launch in airplane mode
- ☐ Language switch: English, Amharic, Afaan Oromo
- ☐ Midnight rollover (set the phone to 23:58 and wait)
- ☐ Time-zone change and back
- ☐ Reboot, then wait for the next reminder
- ☐ Forced Doze, then a reminder
- ☐ Notifications denied; exact alarms denied; battery saver on
- ☐ Two people on one phone
- ☐ Backup → uninstall → reinstall → restore
- ☐ Large text with dark mode
- ☐ TalkBack: mark a dose, record an injection, open Get help
- ☐ Back gesture from every panel
- ☐ App killed while saving

B8. Definition of done for 4.0

- ☐ All tests green, old and new
- ☐ Every phase's "Done when" met and reported
- ☐ Device matrix and scenarios complete
- ☐ Release build: no CAMERA permission; no INTERNET permission, or a written reason
- ☐ Signed release bundle builds on Amin's laptop
- ☐ Play drafts in docs/p1ay/; nothing submitted by Claude
- ☐ Safety charter intact; NEEDS-CLINICAL-REVIEW.md handed to Amin

Appendix A – CLAUDE.md (create exactly this file)

Claude Code loads CLAUDE.md from the repository root at the start of every session, so these rules follow every phase. Keep it short; this version is about 50 lines.

```
# Cardiac Companion – rules for Claude Code

## What this is
Offline Android app (Capacitor 8, Android only, Google Play only) for heart patients in
Ethiopia: warfarin and INR, monthly benzathine penicillin, medicines, weight, symptoms,
warning signs and a visit summary. English, Amharic, Afaan Oromo. Ethiopian and Gregorian
calendars and clocks. Owner: Amin. Work order: docs/claude-code-order.pdf (phases 0-9).
Reference: docs/app-guide.pdf (the v3.2 guide, with a file-by-file line index).

## Safety charter (never break)
1. Never compute, suggest, round or change any dose. Show only what the user entered.
2. No interpretation beyond v3.2: INR vs the doctor's range, the fixed warning-sign
   texts, and the documented weight, symptom and mood rules.
3. Health data never leaves the phone, except files the user chooses to share.
   No servers, accounts, analytics, ads, crash reporters, remote code or remote fonts.
4. Health wording is frozen in all three languages. Propose changes only in
   docs/NEEDS-CLINICAL-REVIEW.md and wait for Amin.
5. Never lose user data. Storage changes need a tested, lossless migration and keep
   the previous copy.
6. Dates use the phone's local calendar day, never UTC.
7. Camera pulse is off in release builds; the release has no CAMERA permission.
8. Suspected bug in medical logic: don't fix it silently. Log it in
   docs/android/FOUND-BUGS.md and ask Amin.

## How to work
- One phase per branch: phase-N-short-name. Plan first; wait for approval.
- Run `npm test` before every commit. Never skip or weaken a test to make it pass.
- Write tests before moving medical logic. Behaviour must not change unless the
  phase says so.
- Ask Amin before: a new dependency, changing the application ID or storage format,
  removing a feature, changing a design rule, anything about signing.
- Never read, print or commit keystores, passwords or keystore.properties.
- Keep en / am / or complete (tests/i18n.test.js) and the flat design rules
  (tests/look.test.js).
- From Phase 2: UI code calls www/js/platform/*, never Capacitor plugins directly.
- Explain in plain English and comment the why, not just the what.
- End each phase with: what changed, phone test steps, risks, decisions needed.

## Commands
- npm test                all Node tests
- npx cap sync android    copy www/ into the Android project
- npx cap open android     open in Android Studio
- cd android && ./gradlew assembleDebug  debug APK (gradlew.bat on Windows)
- cd android && ./gradlew bundleRelease  release bundle (signing set up by Amin)
```

Appendix B — Prompts to paste

Replace N and anything in square brackets. Paste one prompt at a time.

Prompt 0 — the very first session

Read docs/claude-code-order.pdf. It is long: read it in page ranges (at most 20 pages per read), starting with the contents on page 2. Then:

1. Create CLAUDE.md in the repository root with exactly the text in Appendix A.
2. Read docs/app-guide.pdf the same way.
3. In plan mode, propose your plan for Phase 0 only. Change no app code yet.

Prompt for each phase (1 to 9)

Start Phase N of docs/claude-code-order.pdf. Re-read CLAUDE.md, then find the Phase N pages in the contents (page 2) and read them in full. Plan first and wait for my approval. Work on a new branch phase-N-[short-name]. Run npm test before every commit. When the phase is done, stop and give me: what changed, step-by-step tests to do on my Samsung, the risks, and anything I need to decide.

When a phone test fails

On my phone, step [X] failed. I did: [...]. I expected: [...]. I saw: [...] (screenshot attached). Find the cause before changing anything and explain it in plain English. Then fix it, and add a test that would have caught it.

When a session gets long

Summarise where we are in Phase N in ten lines, add it to docs/android/PROGRESS.md, then continue from there.

To learn the code yourself (no changes)

Explain how [the Rosendaal time-in-range code / the Ethiopian date conversion / the reminder scheduler] works, step by step, as if I were a first-year engineering student. Then ask me three questions to check I understood. Don't change any code.

Appendix C – Test vectors

Calculated two independent ways – a Julian Day Number algorithm and the “New Year on 11 or 12 September” rule – which agree on every day from 1901 to 2099. Month 13 is Pagume. Compare month numbers, not spellings.

Gregorian → Ethiopian date

Gregorian	Day	Ethiopian (Y-M-D)	Reads as	What it checks
2023-09-11	Mon	2015-13-06	Pagume 6, 2015	Last day of an Ethiopian leap year
2023-09-12	Tue	2016-01-01	Meskerem 1, 2016	New Year on 12 Sep (2024 is a leap year)
2024-01-01	Mon	2016-04-22	Tahsas 22, 2016	Gregorian new year
2026-01-07	Wed	2018-04-29	Tahsas 29, 2018	Ordinary year
2026-09-10	Thu	2018-13-05	Pagume 5, 2018	Pagume has 5 days in a common year
2026-09-11	Fri	2019-01-01	Meskerem 1, 2019	New Year on 11 Sep
2026-10-03	Sat	2019-01-23	Meskerem 23, 2019	Today, matches the app guide
2026-10-22	Thu	2019-02-12	Tikimt 12, 2019	Matches the guide's example
2027-01-07	Thu	2019-04-29	Tahsas 29, 2019	Before the leap day
2027-09-10	Fri	2019-13-05	Pagume 5, 2019	
2027-09-11	Sat	2019-13-06	Pagume 6, 2019	Leap year: Pagume 6 exists
2027-09-12	Sun	2020-01-01	Meskerem 1, 2020	New Year moves to 12 Sep
2028-02-29	Tue	2020-06-21	Yekatit 21, 2020	Gregorian leap day
2028-09-11	Mon	2021-01-01	Meskerem 1, 2021	Back to 11 Sep
2029-01-01	Mon	2021-04-23	Tahsas 23, 2021	

Ethiopian clock (hours counted from 06:00)

International	Ethiopian	Period
06:00	12	morning
07:00	1	morning
08:00	2	morning
11:59	5:59	morning
12:00	6	day
13:30	7:30	day

International	Ethiopian	Period
18:00	12	evening
21:15	3:15	evening
00:00	6	night
03:00	9	night
05:59	11:59	night

Rule: hour = (h + 6) mod 12, shown as 12 when it is 0. Periods as in the guide: morning 06–12, day 12–18, evening 18–24, night 00–06. Check the app's exact wording separately.

Orthodox Easter and the 55 days before it

Year	Easter (Fasika)	First fasting day (Easter - 55)	Last fasting day (Easter - 1)
2026	Sun 12 Apr	Mon 16 Feb	Sat 11 Apr
2027	Sun 2 May	Mon 8 Mar	Sat 1 May
2028	Sun 16 Apr	Mon 21 Feb	Sat 15 Apr
2029	Sun 8 Apr	Mon 12 Feb	Sat 7 Apr
2030	Sun 28 Apr	Mon 4 Mar	Sat 27 Apr
2031	Sun 13 Apr	Mon 17 Feb	Sat 12 Apr
2032	Sun 2 May	Mon 8 Mar	Sat 1 May

Easter dates were checked with two algorithms. If the app's code defines the fasting window differently (for example, including Easter day), test the app's own definition and flag the difference to Amin instead of changing it. For Ramadan, compute expected dates inside the test with Node's Intl Islamic (Umm al-Qura) calendar; the date announced in Ethiopia can differ by a day, which is why the app asks the user.

Appendix D — Amin's Google Play checklist

These are your jobs, not Claude's. Policies change, so read the current Play Console Help pages as you go.

- 1. Find the publishing organization.** Google requires health apps such as medical apps to be published from an organization account. The organization needs a D-U-N-S number (Dun & Bradstreet issues them free on request, though it can take weeks) and a website verified in Google Search Console.
- 2. Agree the basics in writing:** free, no ads, no data collection, open code, you credited as developer, who handles updates, and how the app is transferred if the partnership ends.
- 3. The organization creates the Play Console account** (a one-time registration fee, US\$25 at the time of writing), completes verification, and invites you as a user with release permissions.
- 4. Create the app:** name, default language, free, app (not game), category Medical.
- 5. App content:** privacy policy URL; app access (no login); ads (none); content rating; target audience (adults); Data safety; Health apps declaration (Medical → Medication and treatment management; Diseases and conditions management; not a medical device).
- 6. Store listing:** texts with the required “not a medical device” sentence, screenshots made with demo data only, feature graphic, icon and a contact email.
- 7. Release in stages:** internal testing (you and your clinician reviewers) → closed testing → production with a staged rollout. Read the pre-launch report every time.
- 8. Guard the upload key:** two offline copies. If it's lost, Google has to reset it, which takes time.

Never

Never declare the app as non-medical to avoid the organization-account rule, and never let a real patient use a build before a clinician has reviewed its health wording.